FLOCK ENERGY

Urja Meter Ops REST API

Complete Project Guide + Source Code

Python 3.10+ · FastAPI · HTTPX · BeautifulSoup · OpenAPI 3.x

Important: This package is a starter implementation. The live portal's actual authentication flow, endpoint paths, selectors, and response schemas must be verified in browser DevTools. The documentation deliberately marks those findings as unverified rather than fabricating them.

Generated project files are also provided separately as a ZIP archive for local editing and GitHub submission.

1. Project Overview

This project wraps a legacy web portal with a clean REST API so downstream clients can retrieve meter listings, individual meter details, and consumption history without manually navigating the portal.

Design: FastAPI exposes the public routes; UrjaPortalClient owns HTTP session cookies, login attempts, re-authentication, upstream requests, and basic HTML/JSON parsing. Pydantic models describe selected response contracts. FastAPI publishes interactive docs at /docs and a schema at /openapi.json.

Read-only scope: no upstream write actions are implemented. The hierarchy route is an explicit 501 placeholder until the real data model is discovered.

2. API Endpoints

Method	Route	Purpose
GET	/health	Local service health check
GET	/api/v1/meters	List meters from upstream
GET	/api/v1/meters/{meter_id}	Retrieve meter detail
GET	/api/v1/meters/{meter_id}/consumption	Retrieve consumption history
GET	/api/v1/hierarchy	Optional extension; currently not implemented

3. Setup and Run

```
python -m venv .venv
# Windows: .venv\\Scripts\\Activate.ps1
# macOS/Linux: source .venv/bin/activate
pip install -r requirements.txt
# Set PORTAL_USERNAME and PORTAL_PASSWORD in your shell
uvicorn app.main:app --reload
```

Then open http://127.0.0.1:8000/docs. Keep secrets in environment variables or an ignored .env file; never commit credentials, cookies, tokens, or real customer data.

4. Discovery Workflow

• Open the portal in Chrome or Firefox, open DevTools → Network, enable Preserve log, and log in using authorized credentials.

• Record the login page request, submission method/path, field names, CSRF behavior, redirect chain, cookie names/attributes, and how success/failure appears. Never record secret values.

• Navigate to meter list, detail, and consumption pages. Identify XHR/fetch calls first; a verified JSON endpoint is preferable to HTML scraping.

• Inspect pagination, filters, date ranges, units, time zones, missing values, and hierarchy references. Save sanitized response fixtures and use them in automated tests.

• Update app/client.py and PROTOCOL.md with observed details. Run tests and regenerate openapi.json from the live FastAPI app before submission.

5. Design Decisions and Trade-offs

FastAPI was chosen for typed route definitions and automatic OpenAPI generation. HTTPX provides cookie persistence and configurable timeouts. BeautifulSoup is used only when upstream responses are HTML.

The adapter boundary isolates legacy protocol behavior from API routes. A 502 response communicates that the wrapper could not obtain upstream data. Generic extraction is intentionally conservative until the real portal schema is verified.

This starter uses a single shared client for simplicity. Production deployment should review concurrency, per-user session boundaries, lifecycle, rate limits, retries, observability, and credential rotation. Caching and database indexing are omitted until freshness and scale requirements are understood.

6. Reflection Template

Complete REFLECTION.md with your own experience. The prompts are: What assumptions did you make? Which part was most difficult and how did you get unstuck? What would you improve with another day? What mistake did you make? What would you criticise in your own submission? Do not claim portal observations or tests you did not perform.

7. Source Code Appendix

The following appendix includes the project source and key documentation files. The ZIP archive contains the editable directory tree.

app/config.py

```
import os
from dataclasses import dataclass

@dataclass(frozen=True)
class Settings:
    portal_base_url: str = os.getenv("PORTAL_BASE_URL", "https://urja-ops.flockenergy.tech").rstrip("/")
    portal_username: str = os.getenv("PORTAL_USERNAME", "")
    portal_password: str = os.getenv("PORTAL_PASSWORD", "")
    request_timeout: float = float(os.getenv("REQUEST_TIMEOUT", "15"))
    # Set only after confirming the actual portal paths in DevTools.
    login_path: str = os.getenv("PORTAL_LOGIN_PATH", "/login")
    meters_path: str = os.getenv("PORTAL_METERS_PATH", "/meters")

settings = Settings()
```

app/client.py

```
"""HTTP adapter for the legacy Urja portal.

IMPORTANT: paths and HTML selectors are deliberately configurable/illustrative.
Confirm them against the live portal using browser DevTools before relying on them.
"""
from __future__ import annotations
from urllib.parse import urljoin
import httpx
from bs4 import BeautifulSoup
from app.config import settings

class PortalError(RuntimeError):
    pass

class PortalAuthenticationError(PortalError):
    pass

class UrjaPortalClient:
    def __init__(self, base_url: str | None = None, username: str | None = None,
                 password: str | None = None, timeout: float | None = None):
        self.base_url = (base_url or settings.portal_base_url).rstrip("/") + "/"
        self.username = settings.portal_username if username is None else username
        self.password = settings.portal_password if password is None else password
        self.login_path = settings.login_path
        self.meters_path = settings.meters_path
        self.http = httpx.Client(
            base_url=self.base_url,
            timeout=timeout or settings.request_timeout,
            follow_redirects=True,
            headers={"User-Agent": "flock-energy-api/1.0"}
        )
        self._authenticated = False

    def close(self) -> None:
        self.http.close()

    def __enter__(self):
        return self

    def __exit__(self, *args):
        self.close()

    def login(self) -> None:
        if not self.username or not self.password:
            raise PortalAuthenticationError(
                "Portal credentials are not configured. Set PORTAL_USERNAME and PORTAL_PASSWORD."
            )
        # Many portals require a GET first to obtain a CSRF token/cookie.
        login_page = self.http.get(self.login_path)
        login_page.raise_for_status()
        soup = BeautifulSoup(login_page.text, "html.parser")
        payload = {"username": self.username, "password": self.password}
        for field in soup.select('input[type="hidden"][name]'):
            payload[field.get("name")] = field.get("value", "")
        # Confirm field names, CSRF handling, and success condition in DevTools.
        response = self.http.post(self.login_path, data=payload)
        response.raise_for_status()
        if self._looks_like_login_page(response.text):
            raise PortalAuthenticationError(
                "Login may have failed or the login flow differs from the configured example."
            )
        self._authenticated = True

    @staticmethod
    def _looks_like_login_page(html: str) -> bool:
        soup = BeautifulSoup(html, "html.parser")
        password_inputs = soup.select('input[type="password"]')
        return bool(password_inputs) and any(
            word in soup.get_text(" ", strip=True).lower()
            for word in ("sign in", "log in", "login", "password")
        )

    def _ensure_login(self) -> None:
        if not self._authenticated:
            self.login()

    def _get(self, path: str) -> httpx.Response:
        self._ensure_login()
        response = self.http.get(path)
        if response.status_code in (401, 403) or self._looks_like_login_page(response.text):
            self._authenticated = False
            self.login()
            response = self.http.get(path)
        response.raise_for_status()
        if self._looks_like_login_page(response.text):
            raise PortalAuthenticationError("Portal session expired and re-login did not succeed.")
        return response
```

```
@staticmethod
def _clean(text: str | None) -> str:
    return " ".join((text or "").split())

def list_meters(self) -> list[dict]:
    response = self._get(self.meters_path)
    # If the portal has a JSON endpoint, prefer it over scraping HTML.
    content_type = response.headers.get("content-type", "").lower()
    if "application/json" in content_type:
        data = response.json()
        if isinstance(data, list):
            return [self._normalise_meter(item) for item in data if isinstance(item, dict)]
        if isinstance(data, dict) and isinstance(data.get("items"), list):
            return [self._normalise_meter(item) for item in data["items"] if isinstance(item, dict)]
        raise PortalError("Unexpected JSON structure for meter list.")
    soup = BeautifulSoup(response.text, "html.parser")
    table = soup.select_one("table")
    if table is None:
        raise PortalError("Could not find a meter table. Confirm portal markup/selectors in
PROTOCOL.md.")
    rows = []
    for tr in table.select("tbody tr"):
        cells = [self._clean(td.get_text(" ", strip=True)) for td in tr.select("th, td")]
        if not cells:
            continue
        # Generic fallback mapping; replace with confirmed headers/columns.
        rows.append({"raw_cells": cells})
    return rows

@staticmethod
def _normalise_meter(item: dict) -> dict:
    """Map known portal fields to the public contract once observed."""
    return {str(k).strip(): v for k, v in item.items()}

def get_meter(self, meter_id: str) -> dict:
    # Replace this path with the real route discovered in DevTools.
    response = self._get(f"{self.meters_path.rstrip('/')}/{meter_id}")
    if "application/json" in response.headers.get("content-type", "").lower():
        data = response.json()
        if isinstance(data, dict):
            return self._normalise_meter(data)
        raise PortalError("Unexpected JSON structure for meter detail.")
    soup = BeautifulSoup(response.text, "html.parser")
    title = self._clean(soup.title.get_text() if soup.title else "")
    fields = {}
    for row in soup.select("tr"):
        cells = [self._clean(c.get_text(" ", strip=True)) for c in row.select("th, td")]
        if len(cells) >= 2:
            fields[cells[0].rstrip(":")] = cells[1]
    return {"meter_id": meter_id, "page_title": title, "attributes": fields}

def get_consumption(self, meter_id: str) -> list[dict]:
    # This is a candidate path only; verify the actual consumption route and date params.
    path = f"{self.meters_path.rstrip('/')}/{meter_id}/consumption"
    response = self._get(path)
    if "application/json" in response.headers.get("content-type", "").lower():
        data = response.json()
        if isinstance(data, list):
            return data
        if isinstance(data, dict) and isinstance(data.get("items"), list):
            return data["items"]
        raise PortalError("Unexpected JSON structure for consumption.")
    soup = BeautifulSoup(response.text, "html.parser")
    table = soup.select_one("table")
    if table is None:
        raise PortalError("Could not find consumption table; inspect portal response.")
    output = []
    headers = [self._clean(x.get_text(" ", strip=True)) for x in table.select("thead th")]
    for tr in table.select("tbody tr"):
        cells = [self._clean(x.get_text(" ", strip=True)) for x in tr.select("td")]
        if cells:
            output.append(dict(zip(headers, cells)) if headers else {"raw_cells": cells})
    return output
```

app/models.py

```
from typing import Any
from pydantic import BaseModel, Field

class MeterSummary(BaseModel):
    meter_id: str = Field(..., description="Stable identifier used by the portal")
    serial_number: str | None = None
    status: str | None = None
    attributes: dict[str, Any] = Field(default_factory=dict)

class MeterListResponse(BaseModel):
    items: list[dict[str, Any]]
    count: int

class ConsumptionResponse(BaseModel):
    meter_id: str
    items: list[dict[str, Any]]
    count: int

class HealthResponse(BaseModel):
    status: str
```

app/main.py

```
from contextlib import asynccontextmanager
from fastapi import FastAPI, HTTPException, Path
from app.client import UrjaPortalClient, PortalAuthenticationError, PortalError
from app.models import MeterListResponse, ConsumptionResponse, HealthResponse

@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.portal = UrjaPortalClient()
    yield
    app.state.portal.close()

app = FastAPI(
    title="Flock Energy — Urja Meter Ops API",
    version="1.0.0",
    description="A normalized REST wrapper around the legacy Urja Meter Ops portal. Portal paths must be
    verified before production use.",
    lifespan=lifespan,
)

def portal_or_502(app: FastAPI):
    return app.state.portal

@app.get("/health", response_model=HealthResponse, tags=["operations"])
def health():
    return {"status": "ok"}

@app.get("/api/v1/meters", response_model=MeterListResponse, tags=["meters"])
def list_meters():
    try:
        items = portal_or_502(app).list_meters()
        return {"items": items, "count": len(items)}
    except PortalAuthenticationError as exc:
        raise HTTPException(status_code=502, detail=f"Portal authentication failed: {exc}")
    except PortalError as exc:
        raise HTTPException(status_code=502, detail=str(exc))
    except Exception:
        # Avoid leaking upstream response bodies, credentials, or internal traces.
        raise HTTPException(status_code=502, detail="Unable to retrieve meters from the upstream
    portal.")

@app.get("/api/v1/meters/{meter_id}", tags=["meters"])
def get_meter(meter_id: str = Path(min_length=1, max_length=128)):
    try:
        return portal_or_502(app).get_meter(meter_id)
    except PortalAuthenticationError as exc:
        raise HTTPException(status_code=502, detail=f"Portal authentication failed: {exc}")
    except PortalError as exc:
        raise HTTPException(status_code=502, detail=str(exc))
    except Exception:
        raise HTTPException(status_code=502, detail="Unable to retrieve meter details from the upstream
    portal.")

@app.get("/api/v1/meters/{meter_id}/consumption", response_model=ConsumptionResponse,
    tags=["consumption"])
def get_consumption(meter_id: str = Path(min_length=1, max_length=128)):
    try:
        items = portal_or_502(app).get_consumption(meter_id)
        return {"meter_id": meter_id, "items": items, "count": len(items)}
    except PortalAuthenticationError as exc:
        raise HTTPException(status_code=502, detail=f"Portal authentication failed: {exc}")
    except PortalError as exc:
        raise HTTPException(status_code=502, detail=str(exc))
    except Exception:
        raise HTTPException(status_code=502, detail="Unable to retrieve consumption from the upstream
    portal.")

@app.get("/api/v1/hierarchy", tags=["hierarchy"])
def get_hierarchy():
    raise HTTPException(status_code=501, detail="Optional extension not implemented; discover and
    document hierarchy data first.")
```

requirements.txt

```
fastapi>=0.110,<1.0
uvicorn[standard]>=0.27,<1.0
httpx>=0.27,<1.0
beautifulsoup4>=4.12,<5.0
pydantic>=2.6,<3.0
pytest>=8.0,<9.0
```

.env.example

```
PORTAL_BASE_URL=https://urja-ops.flockenergy.tech
PORTAL_USERNAME=replace-with-your-portal-username
PORTAL_PASSWORD=replace-with-your-portal-password
REQUEST_TIMEOUT=15
PORTAL_LOGIN_PATH=/login
PORTAL_METERS_PATH=/meters
```

.gitignore

```
.venv/
__pycache__/
.pytest_cache/
.env
*.py[cod]
.DS_Store
```

tests/test_health.py

```
from fastapi.testclient import TestClient
from app.main import app

def test_health():
    with TestClient(app) as client:
        response = client.get("/health")
        assert response.status_code == 200
        assert response.json() == {"status": "ok"}
```

PROTOCOL.md

```
# Urja Meter Ops — Protocol Discovery Notes

**Status: investigation worksheet / unverified scaffold.** Fill this file from your own browser DevTools
    observations. Do not present candidate routes below as confirmed facts.

## Environment
- Portal: `https://urja-ops.flockenergy.tech`
- Date inspected: TODO
- Browser/version: TODO
- Account role/permissions: TODO (do not include secrets)
- Network tab Preserve log enabled: TODO

## Authentication
Record the observed flow:
1. Initial page request and redirects: TODO
2. Login method and path: TODO
3. Content type and exact field *names* (never values): TODO
4. CSRF token/cookie behavior: TODO
5. Response status and redirect chain: TODO
6. Cookie names and relevant attributes (redact values): TODO
7. How successful and failed login are distinguished: TODO
8. Session expiry behavior and whether re-login works: TODO

## Discovered endpoints
| Purpose | Method | Path | Query/body fields | Response type | Verified? |
|---|---|---|---|---|---|
| Login page | TODO | TODO | TODO | HTML/other | No |
| Login submit | TODO | TODO | field names only | HTML/JSON/redirect | No |
| Meter list | TODO | TODO | TODO | HTML/JSON | No |
| Meter detail | TODO | TODO | TODO | HTML/JSON | No |
| Consumption | TODO | TODO | date range/pagination? | HTML/JSON | No |
| Hierarchy (optional) | TODO | TODO | TODO | HTML/JSON | No |

## Data shape and anomalies
- Meter identifier and where it appears: TODO
- Stable meter fields: TODO
- Status values and possible variants: TODO
- Consumption date/time format and timezone: TODO
- Units (kWh, Wh, etc.): TODO
- Missing/null/placeholder values: TODO
- Pagination, sorting, filters, or bulk endpoints: TODO
- Nested tables, repeated labels, inconsistent rows: TODO
- Hierarchy levels and parent identifiers: TODO

## Reproduction notes
For each endpoint, record a sanitized request method/path, relevant headers (excluding
    Cookie/Authorization secrets), status code, content type, and a short sanitized response excerpt.
    Never commit credentials, cookie values, bearer tokens, CSRF values, customer PII, or unredacted
    meter data.

## Implementation consequences
- Replace candidate paths and selectors in `app/client.py` with verified values.
- Prefer a supported JSON endpoint over scraping if one is actually observed.
- Add fixture-based tests for the observed response shape.
- Document any endpoint that could not be verified and how the API responds to that limitation.
```

REFLECTION.md

```
# Reflection

> Replace the bracketed prompts with your actual experience. This scaffold intentionally does not invent
    investigation results or claim tests that have not been run.

## What assumptions did you make?
I chose Python and FastAPI because they provide a compact HTTP service, validation, and generated OpenAPI
     documentation with little boilerplate. I treated the portal as read-only and kept upstream-specific
    behavior inside a dedicated adapter. The candidate routes and form fields remain assumptions until
    confirmed in DevTools; I would not treat a successful HTTP status alone as proof of authentication.

## Which part was most difficult, and how did you get unstuck?
[Describe the real obstacle you encountered. Explain what evidence you inspected—request payload,
    redirect chain, cookies, HTML/JSON response, or a reproducible test—and what changed as a result.]

## If you had another day, what would you improve?
I would finish protocol discovery, add fixture-based tests for each observed response, improve typed
    response models and normalization for dates/numbers/units, and add explicit timeout/retry policy and
    structured observability with secrets redacted. I would also decide whether sessions should be pooled
     or isolated per caller and add caching only after defining freshness requirements.

## What mistake did you make while solving this?
[Describe a real mistake, how you noticed it, its impact, and the change you made. Avoid inventing a
    mistake just to fill the section.]

## If you were reviewing your own submission, what would you criticise?
The main limitation of this starter is that the portal routes, selectors, and schema are not yet
    verified. The generic HTML parser is intentionally conservative and does not guarantee a stable
    public data contract. Before production, I would require observed protocol notes, sanitized fixtures,
     stronger tests, session/concurrency review, and a security review.
```

README.md

```
# Flock Energy — Urja Meter Ops API

A small FastAPI service that wraps the read-only Urja Meter Ops portal and exposes a documented REST
    interface. The adapter supports cookie-based sessions, a login attempt, re-authentication after
    common session-expiry signals, and JSON/HTML response handling.

> **Important status note:** This repository is a working scaffold, not a claim that the live portal has
    been reverse-engineered. Portal login fields, routes, selectors, and response shapes in
    `app/client.py` are candidates. Confirm them with browser DevTools and record observed evidence in
    `PROTOCOL.md` before calling this production-ready.

## Requirements
- Python 3.10+
- Network access to the portal
- Authorized portal credentials

## Setup
```bash
python -m venv .venv
# Windows PowerShell: .venv\Scripts\Activate.ps1
# macOS/Linux: source .venv/bin/activate
pip install -r requirements.txt
copy .env.example .env  # Windows; on macOS/Linux use: cp .env.example .env
```
Edit `.env` with the authorized credentials. Install `python-dotenv` and load it in `app/config.py` if
    you want automatic `.env` loading, or export the variables in your shell. Do not commit `.env`.

PowerShell example:
```powershell
$env:PORTAL_USERNAME="operator@urja.local"
$env:PORTAL_PASSWORD="your-authorized-password"
uvicorn app.main:app --reload
```
macOS/Linux:
```bash
export PORTAL_USERNAME="operator@urja.local"
export PORTAL_PASSWORD="your-authorized-password"
uvicorn app.main:app --reload
```

Explore the API
- Interactive OpenAPI UI: `http://127.0.0.1:8000/docs`
- Raw OpenAPI schema: `http://127.0.0.1:8000/openapi.json`
- Health: `GET http://127.0.0.1:8000/health`
- Meter list: `GET http://127.0.0.1:8000/api/v1/meters`
- Meter details: `GET http://127.0.0.1:8000/api/v1/meters/{meter_id}`
- Consumption: `GET http://127.0.0.1:8000/api/v1/meters/{meter_id}/consumption`
- Hierarchy: `GET /api/v1/hierarchy` (explicitly returns 501 until discovered and implemented)

Example:
```bash
curl http://127.0.0.1:8000/api/v1/meters
curl http://127.0.0.1:8000/api/v1/meters/METER_ID
curl http://127.0.0.1:8000/api/v1/meters/METER_ID/consumption
```

Architecture
- `app/client.py`: upstream HTTP/session adapter and HTML/JSON parsing.
- `app/config.py`: environment-driven settings.
- `app/models.py`: response schemas.
- `app/main.py`: REST routes and upstream error translation.
- `PROTOCOL.md`: investigation worksheet; replace candidates with verified observations.
- `openapi.json`: export generated from the running app after confirming the routes.

The adapter is deliberately separate from FastAPI routes so upstream protocol changes do not spread
    through the API layer.

Authentication and security
Credentials are read from environment variables. The adapter keeps cookies in one `httpx.Client`. It
    first GETs the candidate login path to obtain cookies/hidden form fields, then POSTs credentials and
    hidden fields. This is a generic pattern only; verify actual CSRF token names, form field names,
    redirect behavior, and successful-login indicators. Never log passwords, cookies, CSRF tokens, or
    full upstream HTML in production. Use TLS and restrict access to this wrapper if it is deployed.

Assumptions and trade-offs
- The wrapper is read-only; no write operations are exposed.
- FastAPI is chosen for type-friendly routes and automatic OpenAPI generation.
- A single upstream client is sufficient for a local take-home demo. A production service should use a
    session lifecycle strategy appropriate for concurrency and user/session boundaries.
- HTML parsing is a fallback. A verified internal JSON endpoint is preferable where available.
- Generic table extraction returns raw cells/attributes because the actual portal schema must be observed
     before mapping stable public fields.
- Upstream failures are returned as 502 rather than pretending they are client errors.
- No persistent cache, database, background refresh, or hierarchy reconstruction is included in this
    first pass.

Testing
```bash
```
```

```
pytest -q
```
Add unit tests with saved, sanitized HTML/JSON fixtures and mock HTTP responses. Do not make tests depend on the live portal or commit real session cookies.

OpenAPI export
Start the app, then run:
```bash
python -c "import json, urllib.request;
    json.dump(json.load(urllib.request.urlopen('http://127.0.0.1:8000/openapi.json')),
    open('openapi.json','w',encoding='utf-8'), indent=2)"
```
Re-export after route or schema changes. The checked-in file should match the code.

Before submission
1. Inspect login and data requests in DevTools with Preserve log enabled.
2. Verify actual login route, form payload, CSRF behavior, cookies, and success/failure signals.
3. Verify meter list/detail/consumption routes and pagination/filter parameters.
4. Replace placeholder routes/selectors and normalize real fields and numbers/dates.
5. Add sanitized fixtures and adapter tests.
6. Complete `PROTOCOL.md` and `REFLECTION.md` based on what you personally observed.
7. Run locally, export `openapi.json`, commit, and push to a public GitHub repository.
8. Remove credentials from shell history/logs if needed and never commit them.

Reflection
See [`REFLECTION.md`](REFLECTION.md). Edit it honestly after implementation and testing; do not claim unperformed investigation or tests.

openapi.json


```
{
  "openapi": "3.1.0",
  "info": {
    "title": "Flock Energy \u2014 Urja Meter Ops API",
    "version": "1.0.0",
    "description": "Initial contract placeholder. Regenerate from the running FastAPI app before
    submission."
  },
  "paths": {
    "/health": {
      "get": {
        "tags": [
          "operations"
        ],
        "summary": "Health",
        "responses": {
          "200": {
            "description": "Successful Response"
          }
        }
      }
    },
    "/api/v1/meters": {
      "get": {
        "tags": [
          "meters"
        ],
        "summary": "List Meters",
        "responses": {
          "200": {
            "description": "Successful Response"
          },
          "502": {
            "description": "Upstream portal failure"
          }
        }
      }
    },
    "/api/v1/meters/{meter_id}": {
      "get": {
        "tags": [
          "meters"
        ],
        "summary": "Get Meter",
        "parameters": [
          {
            "name": "meter_id",
            "in": "path",
            "required": true,
            "schema": {
              "type": "string"
            }
          }
        ],
        "responses": {
          "200": {
            "description": "Successful Response"
          },
          "502": {
            "description": "Upstream portal failure"
          }
        }
      }
    },
    "/api/v1/meters/{meter_id}/consumption": {
      "get": {
        "tags": [
          "consumption"
        ],
        "summary": "Get Consumption",
        "parameters": [
          {
            "name": "meter_id",
            "in": "path",
            "required": true,
            "schema": {
              "type": "string"
            }
          }
        ],
        "responses": {
          "200": {
            "description": "Successful Response"
          },
          "502": {
            "description": "Upstream portal failure"
          }
        }
```

```
      }
    },
    "/api/v1/hierarchy": {
      "get": {
        "tags": [
          "hierarchy"
        ],
        "summary": "Get Hierarchy (not implemented)",
        "responses": {
          "501": {
            "description": "Not implemented"
          }
        }
      }
    }
  }
}
```